

Foundations of Quantum Computing

–Week 7– Explore– Study

Niel De Beaudrap and Hex Miller-Bakewell

Grover’s algorithm and Shor’s algorithm remain the two best-known quantum algorithms. This is, in part, because the other problems for which quantum algorithms can provide an advantage are highly technical to describe.

One of the problems for which quantum computers are expected to provide a significant advantage is in the simulation of quantum systems [1], for the simple reason that a quantum computer is itself a versatile and programmable quantum system. We might expect then for a quantum computer to be useful for simulation in the same way that classical computers are useful in simulating classical systems. In particular, quantum computers should yield advantages in simulations of chemical compounds [2, 3], which may facilitate the development and production of pharmaceuticals [4]. This is an exciting frontier in the development of quantum algorithms – but the techniques to do so are in their early days and are highly specialised.

This final week, we sketch more general approaches to trying to solve problems with quantum computers. These approaches are very different to what we have seen before: they rest on ideas which come from fundamental physics and do not have provable bounds on how well they perform in general. They are often called **heuristics** or **ansatzes** for this reason. (Here, ‘ansatz’ is a German word which in physics essentially means ‘an educated guess of how to solve a problem’.)

This week’s Study material introduces many new concepts. (If this is not your first module in Quantum Technologies, you will have encountered some of them before; but perhaps not that many.) In doing so, we introduce some technical concepts as well but do not describe them in much depth. None of the topics covered this week are involved in the portfolio assessment: we present them in order to expose you to concepts which you may encounter if you were to pursue a career in practical quantum computing technology. In this week’s Study section you will cover the following topics:

- 7.1 Continuous-time evolution of quantum systems, under a Hamiltonian
- 7.2 The adiabatic algorithm: a different approach to search
- 7.3 Approximate simulation of Hamiltonians
- 7.4 Quantum alternating operator ansatz (QAOA)

What you should hope to take away from this week is a qualitative understanding of these topics, which will help you to understand and make connections between ideas in quantum technologies that you will encounter later.

7.1 Continuous-time evolution of quantum systems, under a Hamiltonian

We have until now described quantum state-vectors as transforming as a result of a unitary transformation, representing some discrete operation which we perform on a register. However, more generally, the state of a system transforms according to a Hermitian operator $\mathbf{H}$, called the **Hamiltonian** of the system. The Hamiltonian determines, for a quantum register (or other physical system), the energy that the different states have.

In principle, every register is always being acted on by **some** Hamiltonian. When we wish to perform a unitary transformation U on a state, we do this by controlling the Hamiltonian $\mathbf{H}$ which acts on the register $\mathbf{q}$ that stores that state. (When we don't want the state to change, we also do this by controlling the Hamiltonian: specifically, by making it 0 or some multiple of I , so that it does not make any distinctions between the different states that $\mathbf{q}$ might be in.) Thus, to realise different unitary transformations at different times, we must change the Hamiltonian $\mathbf{H}(t)$ itself as a function of time.

For a register acted on by a Hamiltonian, the state-vector of the system $|\psi(t)\rangle$ also changes smoothly in time. The way that the system **evolves** – that is, how it changes with time – is then described by a differential equation called the **time-dependent Schrödinger equation**,

$$\frac{d}{dt} |\psi(t)\rangle = -\frac{i}{\hbar} \mathbf{H}(t) |\psi(t)\rangle , \quad (7.1)$$

describing the instantaneous rate of change at any point in time, involving the imaginary unit and a constant $\hbar > 0$. Because the Hamiltonian $\mathbf{H}(s)$ is Hermitian, we can consider its eigen-decomposition:

$$\mathbf{H}(t) = \sum_{k=0}^{2^n-1} a_k(t) |v_k(t)\rangle\langle v_k(t)| . \quad (7.2)$$

Here, the parameters $a_k(t)$ are the eigenvalues of $\mathbf{H}(t)$ and $|v_k(t)\rangle$ are the eigenvectors associated with those eigenvalues and themselves change smoothly with time.

Note on notation:

To any students of physics: the notation used here is for the sake of consistency with the notation for eigenvalues earlier in the module and do not refer to creation/annihilation operators.

Note on terminology:

To everyone else: don't worry about the fact that physicists have something called 'annihilation operators'. We've all managed to survive for this long, so I'm sure that it's all fine and that you can safely ignore the previous note.

Very often in physics, a Hamiltonian is not expressed as a matrix or a spectral decomposition, but as a sum of much simpler matrix terms. For example, one might have a Hamiltonian $\mathbf{H}$ on n qubits, of the form:

$$\mathbf{H} = H_I + H_T , \quad (7.3)$$

where the operators H_I and H_T themselves are sums of terms, such as:

$$H_I = -Z_1 Z_2 - Z_2 Z_3 - Z_3 Z_4 - \cdots - Z_{n-1} Z_n, \quad H_T = X_1 + X_2 + \cdots + X_n. \quad (7.4)$$

Each of the terms in H_I and H_T , describes a way in which some features of a state $|\psi\rangle$, contributes to the energy $E = \langle\psi|\mathbf{H}|\psi\rangle$ of the system with the Hamiltonian $\mathbf{H}$. (Strictly speaking, only the eigenvectors of $\mathbf{H}$ are considered to have definite energy values, with any other states only having ‘expected’ or probabilistic energy values; we gloss over this distinction here.)

- Recall that $-Z_j Z_{j+1}$ is the operator $-(Z \otimes Z)$ applied specifically to qubits j and $j+1$, with I applied to the other qubits. Then $-Z_1 Z_2$ is an operator with eigenvalues ± 1 , where the -1 -eigenvectors are those in which the first two qubits are in the state $|00\rangle$ or $|11\rangle$ – which are the -1 -eigenvectors of $-Z \otimes Z$.
- Similarly, X_j is shorthand for the operator X applied specifically to qubit j and the identity operator applied to the other qubits.
- The energy $E = \langle\psi|\mathbf{H}|\psi\rangle$ of some state $|\psi\rangle$, would then be calculated as:

$$\begin{aligned} E &= \langle\psi|H_I|\psi\rangle + \langle\psi|H_T|\psi\rangle \\ &= \sum_{j=1}^{n-1} \langle\psi|Z_j Z_{j+1}|\psi\rangle + \sum_{k=1}^n \langle\psi|X_k|\psi\rangle \end{aligned} \quad (7.5)$$

consisting of a sum of contributions arising from each of the individual terms of H_I and H_T .

This way of describing Hamiltonians is often helpful, as it allows us to express $\mathbf{H}$ in terms of interactions between individual qubits (or other physical systems). However, it is often difficult to determine important features of the Hamiltonians, such as their eigenvectors and eigenvalues, from this description. (This seems to be an unavoidable problem and in fact could be interpreted as one of the reasons why quantum computers could be more useful for quantum simulation than classical computers.)

7.1.1 Time-independent Hamiltonians

Suppose that the Hamiltonian $\mathbf{H}(t)$ is unchanging over some time interval $0 \leq t \leq T$: that is, it is equal to a particular fixed operator H_0 during that time. Let us write $A_k := -\frac{1}{\hbar}a_k$ and $|v_k\rangle := |v_k(t)\rangle$ for the eigenvalues and eigenvectors of $\mathbf{H}(t) = H_0$ in this time interval. If $\mathbf{q}$ starts in a state $|\psi_0\rangle = |v_k\rangle$ at time $t = 0$, it is possible to show that $|\psi(t)\rangle$ ‘evolves’ with time during this time interval according to the equation:

$$|\psi(t)\rangle = e^{iA_k t} |v_k\rangle. \quad (7.6)$$

That is, the state-vector only changes by a global phase factor, which changes steadily with time and which depends on the eigenvalue a_k of the state $|\psi_0\rangle$.

We can use this to describe how states change which are **not** eigenvectors of the Hamiltonian H , by considering how states can be described as superpositions of eigenvectors of H_0 . Suppose that instead of $|\psi_0\rangle = |v_k\rangle$, the initial state $|\psi_0\rangle$ of $\mathbf{q}$ is described by:

$$|\psi_0\rangle = \sum_{k=0}^{2^n-1} c_k |v_k\rangle \quad (7.7)$$

for some coefficients $c_0, c_1, \dots, c_{2^n-1} \in \mathbb{C}$ such that $|c_0|^2 + \dots + |c_{2^n-1}|^2 = 1$. Then if $\mathbf{H}(t) = H_0$ during the interval $t \in [0, T]$, the state $|\psi(t)\rangle$ of $\mathbf{q}$ evolves according to:

$$|\psi(t)\rangle = \sum_{k=0}^{2^n-1} c_k e^{iA_k t} |v_k\rangle \quad (7.8)$$

during this time. That is, the amplitudes of the state $|\psi(t)\rangle$ remain the same in **magnitude** over time, but the relative phases between the $|v_k\rangle$ -components change. We can summarise this by describing precisely how $|\psi_0\rangle$ transforms, using the eigenvectors of $\mathbf{H}$:

$$U(t) = \sum_{k=0}^{2^n-1} e^{iA_k t} |v_k\rangle\langle v_k|. \quad (7.9)$$

This is a normal matrix, with eigenvalues which are all roots of unity and is therefore a **unitary** transformation. In a sense, this can be described as the reason **why** we use unitary transformations in particular to represent transformations of quantum states: by fixing a Hamiltonian H_0 for $\mathbf{q}$ for some length of time t , we cause the system to change according to a linear transformation whose eigenvectors are the same as H_0 , but whose eigenvalues are complex phases $e^{iA_k t}$, for various values of $A_k \in \mathbb{R}$.

It can be useful (if only for the sake of brevity) to describe the unitary $U(t)$ directly in terms of the Hamiltonian $\mathbf{H}$ which generates it:

$$U(t) = e^{-i\mathbf{H}t/\hbar}. \quad (7.10)$$

Notice in particular that we are ‘exponentiating’ a linear operator! This is a generalisation of the notion of exponentiating a real or complex number. The exponential function $\exp : \mathbb{C} \rightarrow \mathbb{C}$ is defined by an infinite series:

$$e^z = \exp(z) := \sum_{k \geq 0} \frac{z^k}{k!} = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24} + \dots \quad (7.11)$$

where $n! = n \cdot (n-1) \cdot \dots \cdot 2 \cdot 1$ is the factorial function. Notice that the right-hand side of Eqn. (7.11) is still well-defined as a formula on square matrices: we may then define the exponential of a square matrix M by:

$$e^M = \exp(M) := \sum_{k \geq 0} \frac{M^k}{k!}. \quad (7.12)$$

In the case of normal matrices, we can think of this even more simply: taking the (complex) exponential $\mathbf{H} \mapsto \exp(-i\mathbf{H}t/\hbar)$ of a Hermitian matrix leaves the eigenvectors $|v_k\rangle$ in the spectral decomposition of $\mathbf{H}$ unchanged, but replaces the eigenvalue $a_k \in \mathbb{R}$ of each one with the complex unit $e^{-ia_k/\hbar}$:

$$\mathbf{H} = \sum_{k=0}^{2^n-1} a_k |v_k\rangle\langle v_k| \quad \longmapsto \quad U(t) = \sum_{k=0}^{2^n-1} e^{-ia_k t/\hbar} |v_k\rangle\langle v_k| . \quad (7.13)$$

7.1.2 Hamiltonians which change only slightly with time

It is difficult to summarise much of what can be said of time-dependent Hamiltonians. However, there are interesting and useful things that can be said about Hamiltonians which change only slightly, or only slowly with time.

This provides us with ways of describing how much a small but uncontrolled change in the Hamiltonian affects the results of a computation. In practise, we may not have perfect control over the Hamiltonian $\mathbf{H}(t)$ of a quantum register $\mathbf{q}$. For instance, we might have $\mathbf{H}(t) = H_0 + P(t)$ for some Hermitian operator $P(t)$ which varies with time.

There is a sense in which $P(t)$ can be ‘small’: that is if for all times t , the eigenvalues $b_k(t)$ of $P(t)$ are small. We may define a notion of size for operators, which generalises the notion of a norm for vectors. For the Hermitian operator $P(t)$ and for any value of t , we write:

$$\|P(t)\| = \max_{|\phi\rangle} \|P(t)|\phi\rangle\| \quad (7.14)$$

taking the maximum over all (unit-norm) state-vectors $|\phi\rangle$. This is the same as the largest value of $|b_k(t)|$. Then, if $\|P(t)\| < \varepsilon$ for some small real number $\varepsilon > 0$, this means that the eigenvalues of $P(t)$ are all close to 0, which means that $P(t)$ itself is close to being the zero operator.

Suppose that we have a Hamiltonian $\mathbf{H}(t)$ which is close to H_0 at all times, meaning that $P(t) := \mathbf{H}(t) - H_0$ is ‘small’ in the sense that $\|P(t)\| < \varepsilon \cdot \hbar$ at all times for some small $\varepsilon > 0$. The eigenvectors of $\mathbf{H}(t)$ may not be precisely the same as the eigenvectors $|v_k\rangle$ of the operator H_0 , but the states $|v_k\rangle$ may be ‘very close’ to being eigenvectors of $\mathbf{H}(t)$. Let us write:

$$W_{h,k}(t) = \frac{i}{\hbar} \langle v_h(t) | P(t) | v_k(t) \rangle . \quad (7.15)$$

If $|\psi(t)\rangle \approx |v_k(t)\rangle$ at some time t (for instance, up to some small difference which has a small norm), then $W_{h,k}(t)$ describes the approximate rate at which some other component $|v_h(t)\rangle$ of $|\psi(t)\rangle$ changes at time t , as amplitude is transferred from the $|v_k\rangle$ -component to the $|v_h\rangle$ -component.

If $W_{h,k}(t)$ changes only slowly, we may think of this rate of change in the $|v_h(t)\rangle$ as being a constant multiple of the $|v_k(t)\rangle$ component. While this is happening, however, the $|v_k(t)\rangle$ -component is accumulating phases with time under the influence of H_0 . The accumulated amplitude in the $|v_h\rangle$ -component is also accumulating phase factors, at a

different rate from the $|v_k\rangle$ -component. Let $|\tilde{v}_k(t)\rangle = e^{iA_k t} |v_k\rangle$ and $|\tilde{v}_h(t)\rangle = e^{iA_h t} |v_h\rangle$, to describe time-dependent eigenvectors of H_0 which also take into account the accumulation of phases due to the action of H_0 as a contribution to the Hamiltonian. We may see the effect of this difference in phase factors by considering:

$$\tilde{W}_{h,k}(t) = \langle \tilde{v}_h(t) | P(t) | \tilde{v}_k(t) \rangle = e^{i(A_k - A_h)t} W_{h,k}(t), \quad (7.16)$$

which describes the rate at which the $|\tilde{v}_k(t)\rangle$ -component of $|\psi(t)\rangle$ contributes amplitude to the $|\tilde{v}_h(t)\rangle$ -component.

If $W_{h,k}(t)$ is nearly constant over some time interval, then $\tilde{W}_{h,k}(t)$ changes only by a time-dependent phase factor. If all of the coefficients $W_{h,k}(t)$ are also small (in particular, if $\|P(t) |v_k\rangle\|$ is small), then the rate at which the $|\tilde{v}_k(t)\rangle$ -component decreases with time as a result of this amplitude transfer is also small. Then, the contributions to the $|\tilde{v}_h(t)\rangle$ -component during this time acts like a sum of (many) roots of unity evenly distributed about a circle, similar to what we considered in Figure 6.1 in the Week 6 Study material. In effect, then, these contributions sum very nearly to zero and only fail to sum to zero depending on how $P(t)$ changes or how quickly the $|\tilde{v}_k(t)\rangle$ -component of $|\psi(t)\rangle$ changes. The larger $|A_k - A_h|$ is, the more quickly the contributions evolve to represent a sum of roots of unity about the circle and the closer the accumulation of these amplitudes come to complete destructive interference over small time scales. Similarly, the smaller $P(t)$ is as an operator, the less significant is the decrease in the amplitude of the $|v_k\rangle$ -component. Then, because the other components $|\tilde{v}_h(t)\rangle$ of $|\psi(t)\rangle$ remain close to zero, we would expect $|\psi(t)\rangle$ to remain close to $|\tilde{v}_k(t)\rangle$. In summary, if we start with $|\psi(0)\rangle \approx |\tilde{v}_k\rangle$, and if:

- $\mathbf{H}(t)$ changes only slightly and slowly from some constant Hamiltonian H_0 ,
- and $|a_k - a_h| = \hbar|A_k - A_h|$ is large enough for all $h \neq k$,

then the other components $|\tilde{v}_h(t)\rangle$ of $|\psi(t)\rangle$ will be nearly zero – so that the state remains very nearly in the state $|v_k\rangle$.

7.2 The adiabatic algorithm: a different approach to search

The basis of the adiabatic algorithm is the way that quantum state-vectors change continuously with time, under the influence of a slowly changing Hamiltonian operator. This is the basis of some of the earliest attempts to devise quantum technologies for commercial use. While the technical success of those attempts are the subject of some debate, the theoretical principle which motivated their development is sound and merits some discussion.

7.2.1 The adiabatic theorem

We can extend what we have seen about slightly varying Hamiltonians, to consider what happens over longer time-scales if $\mathbf{H}(t)$ changes slowly enough with time.

We can summarise this as follows. Rather than considering just some constant Hamiltonian H_0 and all changes from it as being ‘perturbations’, we can consider the way that the eigenvectors and eigenvalues of $\mathbf{H}(t)$ change with time, over an interval $t \in [0, T]$. We may consider states:

$$|\tilde{v}_k(t)\rangle = e^{i(\Phi_k(t) + \Gamma_k(t))} |v_k(t)\rangle, \quad (7.17)$$

where $\Phi_k(t)$ represents the effect of the accumulation of phases at varying rates over the interval $[0, T]$, and $\Gamma_k(t)$ represents the way that phases accumulate due to the continuous change in the eigenvectors $|v_k(t)\rangle$ themselves – i.e., continuous changes in **which** vectors are eigenvectors of the matrix $\mathbf{H}(t)$ – over time. If $\mathbf{H}(t)$ changes slowly enough and always has a large enough gap between eigenvalues $a_k(t)$ and $a_h(t)$, then very little amplitude will transfer from the states $|\tilde{v}_k(t)\rangle$ to any other instantaneous eigenstate $|\tilde{v}_h(t)\rangle$. Then the evolution of state-vectors consists mostly of the accumulation of phases for the different eigenvectors, which we incorporate directly into the definition of the vector $|\tilde{v}_k(t)\rangle$ itself. As a result, a register initialised to the state $|\tilde{v}_k(0)\rangle$ will stay very close to being in the state-vector $|\tilde{v}_k(t)\rangle$ during that interval and will end up in a state close to $|\tilde{v}_k(T)\rangle$.

The aforementioned result is known as the **adiabatic theorem**. We have only given a very broad indication of the ideas behind it, but it can be made precise in multiple ways [5, 6, 7], by describing bounds on how much $|\psi(T)\rangle$ differs from $|\tilde{v}_k(T)\rangle$ depending on the size of the gap between $a_k(t)$ and all other eigenvalues $a_h(t)$, the length of time T and bounds on the rate of change of the Hamiltonian $\mathbf{H}(t)$.

The adiabatic theorem is most often applied for the special case in which we consider the eigenvector $|v_0(t)\rangle$ with the minimum possible eigenvalue $a_0(t)$. This is because the eigenvalues of the Hamiltonian $\mathbf{H}(t)$ correspond to the energy of the state of the system/register q which it acts on. (We refer to the state $|v_0(t)\rangle$ as the **ground state** of the Hamiltonian and all other states as ‘excited’ states.)

7.2.2 The adiabatic algorithm

The adiabatic theorem provides conditions in which the system will stay at all times almost completely in the ground state (the lowest-energy state that it could occupy). From a physical and engineering perspective, this simplifies both some of the analysis and the design of computing equipment: keeping the system under control can be done in part simply by keeping the system at low temperature.

Consider how we might take advantage of this feature of quantum physics to perform a computation:

- We may consider how to represent a computational problem, as a set of constraints on bitstrings that describes what it would mean to solve that problem.
 - This defines a **constraint satisfaction problem**, or CSP, which relates to the original problem.

- We may then consider how to represent those constraints in terms of a Hamiltonian H_f , such that any state $|x\rangle$ which is a ground state of H_f that represents a solution x to the CSP defined by those constraints.
 - This is usually done by translating each constraint on a solution x , to a term which can be included in a Hamiltonian H_f . Each of these terms represents an energy contribution to a computational basis state $|x\rangle$, according to whether or not x satisfies the constraint.
 - We conceive of the energy contributions as ‘penalties’ (and we may actually call them ‘energy penalties’) which a state can incur for not satisfying a constraint. Solutions to the CSP are then ground states of H_f : those that avoid all of the energy penalties.
 - Defining the Hamiltonian H_f does not require us to know what the solutions of the CSP are, as H_f is defined in terms of the constraints of the CSP.
- We then consider some other Hamiltonian H_0 , which has a ground state $|\psi_0\rangle$ which is very easy to prepare, but which is not a computational basis state.
 - We almost always take the uniform superposition, $|u\rangle = |+, +, \dots, +\rangle = \frac{1}{\sqrt{2^n}} \sum_x |x\rangle$ for the state $|\psi_0\rangle$, as it already has some amplitude on the solution states of the CSP. This is the ground-state of a Hamiltonian:

$$H_0 = -X_1 - X_2 - \dots - X_n. \quad (7.18)$$

(Recall that X_j is the Pauli X operator, acting on qubit j .) Then the state $|u\rangle$ has the minimum possible energy for each of the terms $-X_j$ and consequently for the Hamiltonian H_0 as a whole.

- Finally, we consider some time-dependent Hamiltonian $\mathbf{H}(t)$, varying over some time interval $t \in [0, T]$ for some duration T , such that $H(0) = H_0$ and $\mathbf{H}(T) = H_f$.
 - Very commonly, one considers $\mathbf{H}(t) = \frac{t}{T}H_f + (1 - \frac{t}{T})H_0$, interpolating between the two Hamiltonians H_0 and H_f ; but this is just one possibility.

Provided that the time-dependent Hamiltonian $\mathbf{H}(t)$ satisfies the conditions of the adiabatic theorem – also provided that we have enough control over a quantum system $\mathbf{q}$ that we can prepare the state $|\psi_0\rangle$ and realise the Hamiltonian $\mathbf{H}(t)$ on it – we can then solve the original computational problem as follows:

1. Prepare $\mathbf{q}$ in the state $|\psi_0\rangle$.
2. Act on $\mathbf{q}$ with the Hamiltonian $\mathbf{H}(t)$, varying the Hamiltonian with time for a time interval $t \in [0, T]$.
3. Measure $\mathbf{q}$ in the computational basis.

By the adiabatic theorem, the final state before measurement will be a ground state of H_f , which is a solution to the computational problem of interest.

The preceding is called **the adiabatic algorithm**. (Note that this is not the same as the adiabatic theorem – the difference is that the adiabatic **theorem** is a provable result about what happens under certain circumstances and the adiabatic **algorithm** is a procedure to solve problems by taking advantage of that provable result.) It comes with a couple of caveats:

Hamiltonian complexity:

If the final Hamiltonian H_f is too complicated, it may be difficult or impossible to directly realise a Hamiltonian $\mathbf{H}(t)$ such that $\mathbf{H}(T) = H_f$. There are also similar problems with $\mathbf{H}(t)$ for $t \in [0, T]$.

‘Spectral gaps’:

It is not easy to know, in general, whether $\mathbf{H}(t)$ actually satisfies the conditions of the adiabatic theorem. Specifically, the various quantitative versions of the adiabatic theorem all rely on there being a ‘spectral gap’: some known minimum gap between the ground state of $\mathbf{H}(t)$ and the other eigenvalues, for all $t \in [0, T]$. But determining whether this is true is itself a difficult problem.

Despite these issues, the adiabatic algorithm represents an interesting conceptual approach to solving problems with a quantum computer. In practice, one might be able to get past the difficulties in realising H_f by using clever approximations which work to ‘effectively’ realise specific Hamiltonians. Furthermore, even if one cannot guarantee that a Hamiltonian $\mathbf{H}(t)$ has a spectral gap, one can attempt to satisfy the conditions of the adiabatic theorem by transitioning from H_0 to H_f more slowly – for example, letting $\mathbf{H}_1(t) = \mathbf{H}(t/2)$ for $t \in [0, 2T]$, $\mathbf{H}_2(t) = \mathbf{H}(t/4)$ for $t \in [0, 4T]$, and so forth. So long as there is **some** spectral gap, one will eventually find a way to ‘evolve the Hamiltonian’ slowly enough to get a solution to the computational problem with significant probability. In this way, we can still consider how to solve problems with the adiabatic algorithm.

7.3 Approximate simulation of Hamiltonians

Whether to simulate a quantum mechanical system such as a molecule, or to try to simulate a complicated Hamiltonian corresponding to a constraint satisfaction problem (or CSP, as described in the context of the adiabatic algorithm in Section 7.2.2), it may be useful to be able to somehow simulate how a physical system evolves under the influence of a given Hamiltonian $\mathbf{H}$.

If we want to use a quantum computer to simulate the Hamiltonian $\mathbf{H}$ of another quantum mechanical system, or an artificially constructed Hamiltonian corresponding to some CSP, it may be that the Hamiltonians $\tilde{\mathbf{H}}$ that we can use to do this are not ones that can be easily realised through sheer engineering, on a register $\mathbf{q}$ of qubits. However, even if it is difficult to realise $\tilde{\mathbf{H}}$ on $\mathbf{q}$ in **hardware**, we may simulate $\tilde{\mathbf{H}}$ through quantum **software** – or to be more precise, through quantum algorithms written as a sequence of

instructions, such as a QAAL program.

Hamiltonian simulation is an active area of research, with sophisticated techniques (see for example Ref. [2] and works which cite it) which we will not be able to do justice to in this module. We will describe instead one of the simplest techniques, which is also relevant to simulating Hamiltonians with classical computers, called **Trotterisation**.

7.3.1 Product formulas for time-independent Hamiltonians

Suppose that we have a Hamiltonian $\mathbf{H}$ which is expressed as a sum of terms, $\mathbf{H} = H_1 + H_2 + \dots + H_m$. There is a possibility that these terms all have a set of eigenvectors in common. (For example, with the operator H_I described in Eqn. (7.4), every computational basis state $|x\rangle$ is an eigenvector of every single one of the terms. Similarly, for the operator H_T described in Eqn. (7.4), any state which is a tensor product of $|+\rangle$ and $|-\rangle$ states is an eigenvector for each of the terms X_j .) In this case, all of the terms **commute** – that is, we have $H_j H_k = H_k H_j$ for all pairs of terms H_j and H_k . When this happens, we can simulate the effect of the Hamiltonian $\mathbf{H}$ for some amount of time t , by simulating each of the terms H_j independently for time t :

$$e^{-i\mathbf{H}t/\hbar} = e^{-iH_1t/\hbar} e^{-iH_2t/\hbar} \dots e^{-iH_mt/\hbar}. \quad (7.19)$$

If the terms H_j are all simple enough (for example, they act only on a small number of qubits each), we can then realise each of the unitary transformations $e^{-iH_jt/\hbar}$ separately, using a sequence of simple unitary transformations (for example, using a QAAL program).

We are also interested in Hamiltonians $\mathbf{H}$ where not all the terms ‘commute’ with one another in this way, because they do not all have eigenvalues in common. (The Hamiltonian $\mathbf{H}$ in Eqn. (7.3) is an example: the terms in H_I and H_T do not all have a common set of eigenvectors.) In this case, Eqn. (7.19) does **not** hold. However, for small values of t , the difference between the left-hand side and the right-hand side of Eqn. (7.19) becomes very small. By breaking the evolution over time T and breaking it into small time intervals of length $0 < t \ll 1$ for which Eqn. (7.19) is **approximately** true, we may consider approximating evolution under the Hamiltonian $\mathbf{H}$ by:

$$e^{-i\mathbf{H}T/\hbar} \approx \left(e^{-iH_1t/\hbar} e^{-iH_2t/\hbar} \dots e^{-iH_mt/\hbar} \right)^{T/t}. \quad (7.20)$$

This approximation gets better as we take $t \rightarrow 0$. This approach to simulating Hamiltonians, consisting of non-commuting terms, is known as **Trotterisation**. We can obtain better approximations with more sophisticated decompositions, such as:

$$e^{-i\mathbf{H}T/\hbar} \approx \left[\left(e^{-iH_1t/2\hbar} \dots e^{-iH_mt/2\hbar} \right) \left(e^{-iH_mt/2\hbar} \dots e^{-iH_1t/2\hbar} \right) \right]^{T/t}, \quad (7.21)$$

and still more complex decompositions. These more generally are referred to as ‘operator product formulas’ to simulate the Hamiltonian $\mathbf{H}$.

For some operator product formulas, it is possible to precisely quantify the amount of error: analysis of how this might be done is described in Refs. [8, 9]. These more sophisticated approximations generally involve an increasing number of factors $e^{-iH_j\tau}$ for various $\tau > 0$, which will increase the complexity of simulating $\mathbf{H}$. Then, how we should attempt to simulate a Hamiltonian $\mathbf{H}$ depends in part on how close an approximation we want to achieve and how many operations we are willing to perform to achieve that level of accuracy.

One way that we control the amount of error in an operator product formula is to consider what terms **do** commute in the decomposition of $\mathbf{H}$, and to group them together into ‘sub-Hamiltonians’ (such as H_I and H_T in Eqn. (7.3)) that we can more easily simulate using a decomposition such as Eqn. (7.19). If $\mathbf{H} = H_A + H_B$, then have:

$$\left\| e^{-i\mathbf{H}t/\hbar} - \left(e^{-iH_A t/2\hbar} e^{-iH_B t/\hbar} e^{-iH_A t/2\hbar} \right) \right\| \leq \frac{\|A\| \|B\| (\frac{1}{2}\|A\| + \|B\|)}{3\hbar^3} t^3. \quad (7.22)$$

(This is a simplification of the result in Eqn. (L5) of Ref. [8].) From this we can then show:

$$\left\| e^{-i\mathbf{H}T/\hbar} - \left(e^{-iH_A T/2\hbar} e^{-iH_B T/\hbar} e^{-iH_A T/2\hbar} \right)^{T/t} \right\| \leq \frac{\|A\| \|B\| (\frac{1}{2}\|A\| + \|B\|)}{3\hbar^3} T t^2, \quad (7.23)$$

which approaches zero as $t \rightarrow 0$. This remains true even if H_A and H_B themselves decompose into multiple terms each; but if the terms in H_A commute with one another (and similarly for H_B), then we may simulate H_A (and H_B) using a decomposition as in Eqn. (7.19).

7.3.2 Operator product formulas for time-dependent Hamiltonians

We can use the ideas just described, not just to simulate a time-independent Hamiltonian, but also to simulate time-dependent Hamiltonians, as follows:

Consider a Hamiltonian operator $\mathbf{H}(t)$, which changes smoothly with time in an interval $t \in [0, T]$. For any $N > 1$, we can divide this into N intervals,

$$\left[0, \frac{1}{N}T\right], \quad \left[\frac{1}{N}T, \frac{2}{N}T\right], \quad \left[\frac{2}{N}T, \frac{3}{N}T\right], \quad \dots, \quad \left[\frac{N-1}{N}T, T\right]. \quad (7.24)$$

For each interval $[\frac{j}{N}T, \frac{j+1}{N}T]$, we can consider some Hamiltonian $\tilde{\mathbf{H}}_j = \mathbf{H}(t_j)$ from a point in time $t_j = (j + \frac{1}{2})T/N$ mid-way through that interval, or from any other time $t_j \in [\frac{j}{N}T, \frac{j+1}{N}T]$ in that interval. We may consider the unitary transformation U_j that comes from evolving the system according to the time-varying Hamiltonian $\mathbf{H}(t)$ during the interval $[\frac{j}{N}T, \frac{j+1}{N}T]$, and compare this to the unitary transformation $\tilde{U}_j = e^{-i\tilde{\mathbf{H}}_j T/N\hbar}$ which arises from simulating the Hamiltonian $\tilde{\mathbf{H}}_j$ for the same amount of time. For a sufficiently large N (and so for a sufficiently small duration $\frac{1}{N}T$), the Hamiltonian $\mathbf{H}(t)$ does not differ much from $\tilde{\mathbf{H}}_j$ throughout the interval $[\frac{j}{N}T, \frac{j+1}{N}T]$: then $\tilde{U}_j \approx U_j$ as well. Thus, if we can simulate the unitaries $\tilde{U}_j$ in turn, we can approximately simulate the effect of the more general Hamiltonian $\mathbf{H}(t)$ throughout the interval $[0, T]$; and by

taking N large enough, we can reduce the error in this simulation to be as small as we like.

We can take this idea and combine it with the notion of ‘Trotterisation’ or ‘operator product formulas’ described in the preceding section, to simulate the operators $\tilde{U}_j = e^{-i\tilde{\mathbf{H}}_j T/N\hbar}$. If we can decompose $\tilde{\mathbf{H}}_j$ as a sum $H_{j,1} + H_{j,2} + \cdots + H_{j,m_j}$ of operators which we can easily realise exactly, then we can approximate $\tilde{U}_j$ by using some approximation such as the ones in Eqn. (7.20) or (7.21). For a sufficiently large N , we may even be able to use a quantifiable bound such as the one of Eqn. (7.22), to simulate $\tilde{\mathbf{H}}_j$ accurately for a small interval of time.

7.4 Quantum alternating operator ansatz (QAOA)

All of the preceding leads up to one of the most recent techniques to solve difficult problems using quantum computers: the QAOA (which can be taken to stand either for ‘Quantum Alternating Operator Ansatz’ or ‘Quantum Approximate Optimisation Algorithm’). The idea behind the QAOA is:

- to perform a procedure which is in some respects like a Trotterisation or operator product formula approximation of the adiabatic algorithm; but
- not constraining the number of operators in the product, or their durations, to anything for which we have a general error bound such as that in Eqn. (7.22).

We may elaborate as follows:

(a) Combining the adiabatic algorithm with operator product approximations

Generally, to solve an optimisation problem, we are interested in the question of minimising some ‘cost function’ $f : \{0, 1\}^n \rightarrow \mathbb{R}$ – a function which describes how features of some input (such as a bitstring) lead to higher or lower values of some ‘cost’ function. An example might be energy penalties associated with the number of constraints in a CSP which a bitstring $x \in \{0, 1\}^n$ does not satisfy, as we described in Section 7.2.2. More generally, we may also consider the situation in which **no** bitstring x avoids all energy penalties, but where we are still interested in considering what values of x at least **minimise** any contributions to the cost function. We may represent this by a Hamiltonian H_f whose eigenvectors are computational basis states, such that $\langle x | H_f | x \rangle = f(x)$.

Having defined such a Hamiltonian H_f , finding a bitstring $x \in \{0, 1\}^n$ which minimises the cost function $f(x)$ is then equivalent to finding a ground state of H_f . In principle we may do this using the adiabatic algorithm, for instance,

1. preparing some n -qubit register $\mathbf{q}$ in the state $|+, +, \dots, +\rangle$, which is the ground state of the Hamiltonian $H_0 = -X_1 - X_2 - \cdots - X_n$
2. acting on the register $\mathbf{q}$ with the Hamiltonian $\mathbf{H}(t) = \frac{t}{T}H_f + (1 - \frac{t}{T})H_0$, for some interval of time $t \in [0, T]$
3. measuring the register $\mathbf{q}$ in the computational basis.

It is worth noting that, in general, we cannot know whether the result actually does represent the minimum value of $f(x)$. If $f(x)$ represents the energy penalties associated with a CSP, then we can know that we have found the minimum if $f(x) = 0$, as this implies that no constraints are left unsatisfied. Generally, the best we can realistically hope for is that the outcome $x \in \{0, 1\}^n$ has a smaller value for $f(x)$ than we could easily find otherwise. (Often, in optimisation problems, it is still useful to find results which might not be a global minimum, but which just have as low a ‘cost’ as you can manage to find.)

It may be impractical to design a quantum computer with fine enough control over the evolution of $\mathbf{q}$ to act on it with the Hamiltonian $\mathbf{H}(t)$. In particular, the H_f contribution may be too complicated to realise directly. But we can approximate the unitary transformation U that would result from $\mathbf{H}(t)$ over the time interval $t \in [0, T]$, using more general-purpose operations (such as in QAAL programs) to approximate $\mathbf{H}(t)$ using operator product formulas. This could be done by a sequence of unitary transformations,

$$U = e^{-iH_f t_M/\hbar} e^{-iH_0 t_{M-1}/\hbar} \dots e^{-iH_f t_3/\hbar} e^{-iH_0 t_2/\hbar} e^{-iH_f t_1/\hbar} \quad (7.25)$$

alternating between simulations of H_f and H_0 , for various durations of time $t_1, t_2, \dots, t_M > 0$. (The previous formula hints that M may be odd, but we may consider even values of M as well, ending with simulating H_0 .) The specific lengths of time, t_j , can be computed using the techniques described in the preceding section: choosing a value of N for which the Hamiltonians $\tilde{\mathbf{H}}_j$ are good approximations to $\mathbf{H}(t)$ in different time-intervals and then approximating evolution under the Hamiltonians $\tilde{\mathbf{H}}_j$ using Trotterisation or similar techniques. This will depend in part on just how closely we wish to simulate the Hamiltonian $\mathbf{H}(t)$.

In the preceding material, evolution under the Hamiltonian H_0 can be easily simulated as a product of x-axis rotations. Evolution under H_f may be more complicated, but can in principle be reduced to a classical computation (of the cost function f on computational basis state inputs), together with z-axis rotations depending on the results of that computation. (For constraint satisfaction problems in particular, we can separately compute whether each constraint has been violated or not and perform a z-axis rotation depending on each of these results.) Whether or not it is ‘easy’ to simulate the Hamiltonian H_f , we at least know in principle that it is possible, at least for any general-purpose quantum computer which is well-enough developed to perform those computations. This then represents a general approach to solving optimisation problems on a quantum computer.

(b) Generalisation to QAOA

We have just described an approach to solving optimisation problems by simulating alternating Hamiltonians for varying lengths of time. However, if we try to simulate the adiabatic algorithm very closely, the number of alternations M may be very large. The resulting computation might then take a long time.

One may ask whether this is strictly necessary and whether perhaps it is possible to find outcomes $x \in \{0, 1\}^n$ which at least come closer to minimising the cost function f than what we can easily find with classical computers, by reducing the number of alternations, M . This can be motivated practically, for instance, by considering only small values of M (for instance, $M \leq 6$) it is likely to be feasible to realise this for near-term quantum devices. With a small number of alternations, it is unclear how we may simulate the adiabatic algorithm very closely. However, it is not strictly speaking necessary to do so: having established that it is possible to solve optimisation problems with an ‘alternating operator ansatz’ – a sequence of transformations as in Eqn. (7.25) – we may simply experiment with different values for $t_1, t_2, t_3, \dots$, if there are not too many parameters to vary.

This is the basis for **QAOA**: the Quantum Alternating Operator Ansatz, as it was originally called [10], or the Quantum Approximate Optimisation Algorithm as it is now sometimes called. In outline, we can describe QAOA as follows:

1. Choose a value of M .
2. Choose some values of $t_1, t_2, \dots, t_M > 0$.
3. Prepare a register $\mathbf{q}$ of n qubits in the state $|+, +, \dots, +\rangle$.
4. Simulate the operations $e^{-iH_f t_M/\hbar} \dots e^{-iH_0 t_2/\hbar} e^{-iH_f t_1/\hbar}$ on $\mathbf{q}$.
5. Measure $\mathbf{q}$ in the computational basis and record the result.
6. Return either to Step 1 (to vary the number of alternations) or to Step 2 (to vary only the values of t_j) – repeating until either you find a good enough result or you get bored.

There are varying approaches to going about choosing M in Step 1, choosing the times t_j in Step 2 and choosing how many times to repeat the procedure in Step 6. None of these have performance guarantees: in this sense, it is not clear to what extent this is actually an ‘algorithm’. (It is also not the only way to use quantum algorithms to try to find approximately optimal solutions to problems.) But it does represent an interesting and general approach to try to obtain better solutions to optimisation problems, than we can do classically.

QAOA is an example of a class of procedures known as **Variational Quantum Algorithms** [12], or VQAs. These attempt to achieve a quantum advantage in computation, not through a formal algorithm with performance guarantees, but through what is essentially **experimental** computation. In to consider something based on an educated guess (an ‘ansatz’) and to try to discover what results can be found by experimenting with the values of parameters which determine the operations which are performed. While it is difficult to predict how useful the results produced by these **ansatzes** will be in general, they represent a change of approach in algorithm design, which may prove useful and productive when learning what quantum computers can do in a setting different from a chalkboard or whiteboard.

Summary

In this week, we aimed to present a series of technical concepts at a very high level, to introduce you to potentially important topics of a very different sort to what we have seen in the rest of the module. With some luck, you will have obtained an impression of:

- the connection between Hamiltonians and unitary transformations, through the Schrödinger equation
- the concept of operator norms, as a way of describing how close two Hamiltonians are to one another, or how close two unitary transformations are to one another
- the adiabatic theorem and the adiabatic algorithm, as a major departure from the notion of quantum computation performed by sequences of unitary operations
- the way that we can in any case approach simulating continuous Hamiltonians by Trotterisation and similar techniques, using sequences of unitary transformations
- the QAOA, as an experimental approach to solving problems, acting as a simplification of how we might simulate the adiabatic algorithm through Trotterisation.

These topics will help you to better appreciate what may be possible, with quantum computing technology.

References

- [1] Childs, A. and Wiebe, N. Hamiltonian simulation using linear combinations of unitary operations, *Quantum Information and Computation*, 2012. [Online], 12, pp. 901–924. Available at: <https://arXiv.org/abs/1202.5822> [accessed 7 September 2023].
- [2] Aspuru-Guzik, A., Dutoi, A. D., Love, P. J. and Head-Gordon, M. Simulated quantum computation of molecular energies. *Science*, 2005. [Online], 309, pp. 1704–1707. Available at: <https://arXiv.org/abs/quant-ph/0604193> [accessed 7 September 2023].
- [3] Gharibian, S. and Le Gall, F. Dequantizing the Quantum singular value transformation: hardness and applications to Quantum chemistry and the Quantum PCP conjecture. *Proceedings 54th Annual STOC*, 2022, pp. 19–32.
- [4] Santagati, R., Aspuru-Guzik, A., Babbush, R., Degroote, M., Gonzalez, L., Kyoseva, E., Moll, N., Oppel, M., Parrish, R. M., Rubin, N. C., Streif, M., Tautermann, C. S., Weiss, H., Wiebe, N., Utschig-Utschig, C. Drug design on quantum computers. *Cornell e-print archive*, 2023. [Online]. doi: 10.48550/arXiv.2301.04114 [accessed 7 September 2023].
- [5] Reichardt, B. The quantum adiabatic optimization algorithm and local minima. *Proceedings 36th STOC*, 2004, pp. 502–510.
- [6] Ambainis, A. and Regev, O. An Elementary Proof of the Quantum Adiabatic Theorem. *Cornell e-print archive*, 2004. [Online]. Available at: <https://arXiv.org/abs/quant-ph/0411152> [accessed 07 September 2023].

- [7] Jansen, S., Ruskai, M.-B. and Ruedi, S. Bounds for the adiabatic approximation with applications to quantum computation. *Journal of Mathematical Physics*, 2007. [Online], 48, p. 102111. Available at: <https://arXiv.org/abs/quant-ph/0603175> [accessed 7 September 2023].
- [8] Childs, A. M., Su, Y., Tran, M. C., Wiebe, N. and Zhu, S. Theory of Trotter Error with Commutator Scaling. *Physical Review*, 2021, X (11), p. 011020.
- [9] Low, G. H., Kliuchnikov, V. and Wiebe, N. Well-conditioned multiproduct Hamiltonian simulation. *Cornell e-print archive*, 2019. [Online]. doi: 10.48550/arXiv.1907.11679 [accessed 7 September 2023].
- [10] Farhi, E., Goldstone, J. and Gutmann, S. A Quantum Approximate Optimization Algorithm. *Cornell e-print archive*, 2014. [Online] doi: 10.48550/arXiv.1411.4028 [accessed 7 September 2023].
- [11] Blekos, K., Brand, D., Ceschini, A., Chou, C.-H., Li, R.-H., Pandya, K. and Summer, A. A Review on Quantum Approximate Optimization Algorithm and its Variants. *Cornell e-print archive*, 2023. [Online]. doi: 10.48550/arXiv.2306.09198 [accessed 7 September 2023].
- [12] Cerezo, M., Arrasmith, A., Babbush, R., Benjamin, S. C., Endo, S., Fujii, K., McClean, J. R., Mitaraim, K., Yuan, X., Cincio, L. and Coles, P. J. Variational Quantum Algorithms. *Nature Reviews Physics*, 2021, [Online], 3, pp. 625–644. doi.org/10.1038/s42254-021-00348-9 [accessed 07 September 2023].